

# Machine Learning Algorithms Laboratory

## Experiment 6 Report

|                  |                                                                |
|------------------|----------------------------------------------------------------|
| Experiment No.   | 6                                                              |
| Experiment Title | Bagging, Boosting, and Stacked Ensemble Models<br><sup>1</sup> |
| Student Name     | Gautham S                                                      |
| Register Number  | 3122247001016                                                  |
| Faculty          | Dr. Poreddy Ajay Kumar Reddy                                   |
| Submission Date  |                                                                |

## 1 Objective

- To understand ensemble learning strategies: Bagging, Boosting, and Stacking.
- To implement Bagging and Boosting classifiers.
- To build a Stacked Ensemble model using multiple base learners.
- To compare ensemble models in terms of accuracy, stability, and generalization.
- To analyze the effect of ensemble methods on bias and variance.

## 2 Theory

### 2.1 Bagging (Bootstrap Aggregation)

Bagging trains several copies of the same base model on different bootstrap samples (random samples drawn with replacement) of the training data, and combines their predictions by voting/averaging. Since each model sees a slightly different version of the data, their errors tend to be different from each other, and averaging them out reduces the overall variance. Bagging works best with high variance, unstable base models like a fully grown decision tree.

### 2.2 Boosting

Boosting trains models sequentially, where each new model tries to correct the mistakes made by the previous ones, usually by giving more weight/importance to the samples that were misclassified earlier. This is different from bagging because the models are not trained independently, they build on top of each other. Boosting mainly reduces bias, since it keeps improving on the weak points of the earlier models. AdaBoost and Gradient Boosting are two common boosting algorithms.

## 2.3 Stacked Ensemble

Stacking combines predictions from multiple different (heterogeneous) base models by training a separate meta-learner that learns how to best combine their outputs. Unlike bagging/boosting which usually use the same type of base model, stacking benefits from using different kinds of models (for example SVM, Naive Bayes and Decision Tree) since each one may capture a different aspect of the data, and the meta-learner can learn which model to trust more in which situation.

## 3 Dataset Description

| Dataset            | Description                                     |
|--------------------|-------------------------------------------------|
| Dataset Name       | Wisconsin Diagnostic Breast Cancer Dataset      |
| Dataset Source     | UCI Machine Learning Repository                 |
| Number of Samples  | 569                                             |
| Number of Features | 30 numerical features                           |
| Target Classes     | Benign (0), Malignant (1)                       |
| Missing Values     | None                                            |
| Class Distribution | Benign: 357 (62.7%), Malignant: 212 (37.3%)     |
| Train-Test Split   | 80% Train (455 samples), 20% Test (114 samples) |

## 4 Exploratory Data Analysis

The Diagnosis column ("B"/"M") was mapped to 0 (Benign) and 1 (Malignant) before training. The ID column was dropped since it is not a useful feature.

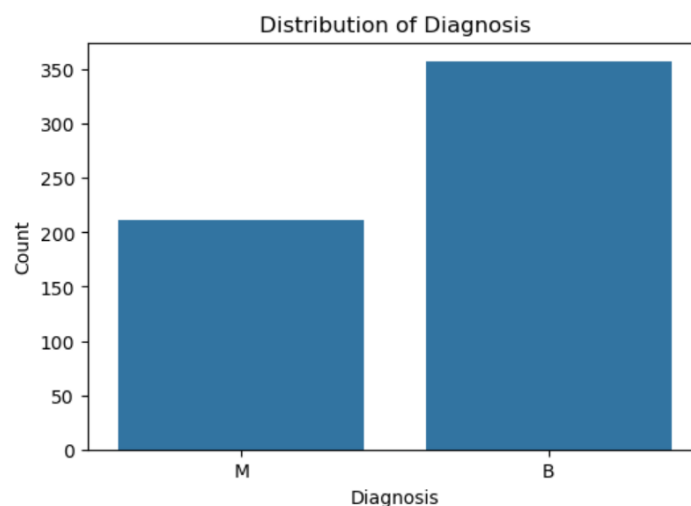


Figure 1: Distribution of Diagnosis

**Inference:** The dataset has more benign cases (357) than malignant (212), a moderate class imbalance. Because of this, precision, recall and F1-score were checked in addition to accuracy so that the minority (malignant) class performance is not hidden by the overall accuracy.

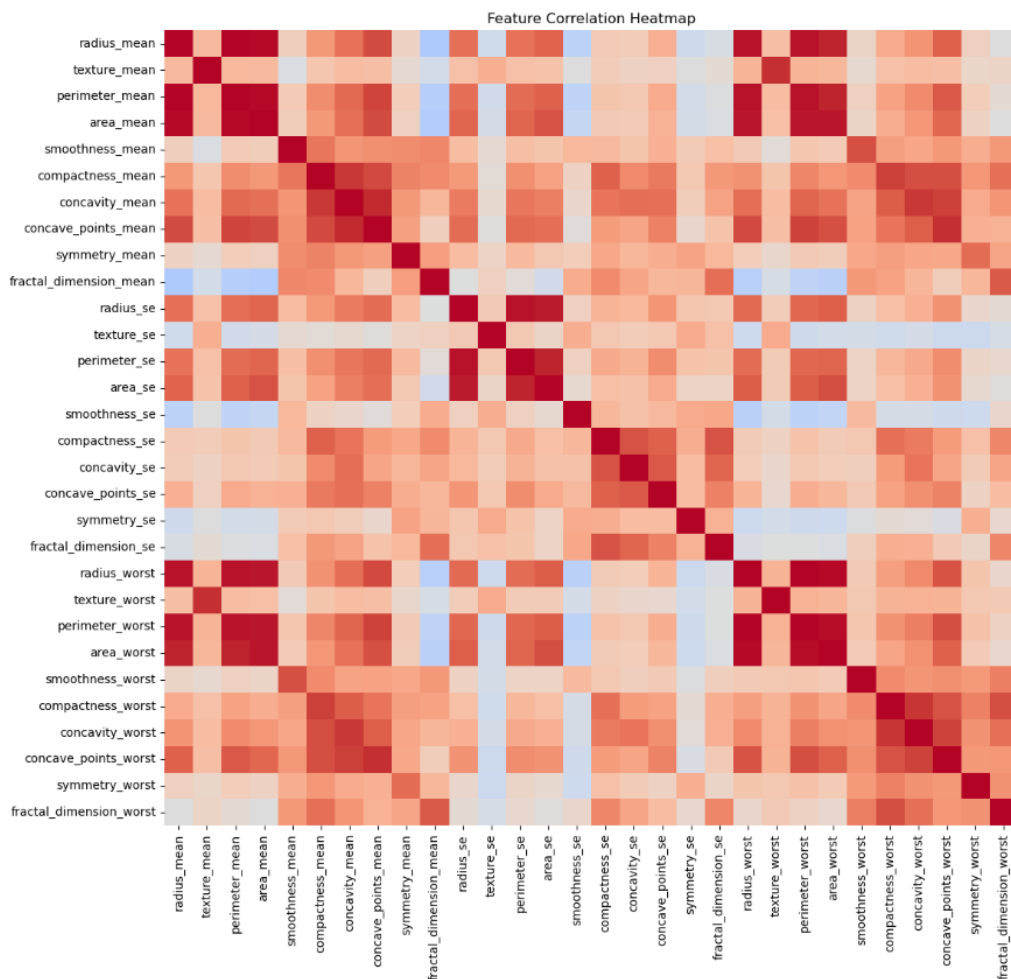


Figure 2: Feature Correlation Heatmap

**Inference:** A lot of the features are highly correlated, for example the mean, standard error and worst versions of radius/perimeter/area all move together since they measure related aspects of the tumor's size. On checking a quick group-wise mean comparison, malignant tumors were also found to have noticeably higher radius, perimeter, area and compactness on average compared to benign ones.

## 5 Model Implementation

### 5.1 Bagging

A Bagging Classifier was built using a Decision Tree as the base estimator, and the number of estimators was tuned using 5-fold cross validation.

```
bagging = BaggingClassifier(
    estimator=DecisionTreeClassifier(random_state=42),
    random_state=42
)

for n in [10, 25, 50, 100]:
    model = BaggingClassifier(estimator=DecisionTreeClassifier(
        random_state=42),
```

```

n_estimators=n, max_samples=0.8,
max_features=1.0, random_state=42)
scores = cross_validate(model, X_train, y_train, cv=cv,
    scoring=["accuracy", "f1"])
# store mean accuracy and f1 for each n

```

| n_estimators | max_samples | Avg CV Accuracy (%) | Avg CV F1 Score (%) |
|--------------|-------------|---------------------|---------------------|
| 10           | 0.8         | 94.51               | 92.26               |
| 25           | 0.8         | 96.92               | 95.85               |
| 50           | 0.8         | 96.48               | 95.26               |
| 100          | 0.8         | 95.60               | 93.99               |

Table 2: Bagging Hyperparameter Evaluation

**Inference:** Accuracy peaked at `n_estimators=25` (96.92%) and actually went down a bit as more trees were added (100 estimators gave only 95.60%). This shows that after a certain number of trees, adding more of them does not necessarily help and might even add a bit of noise, so more estimators does not always mean a better model.

## 5.2 Boosting

Both AdaBoost (with decision stumps, `max_depth=1`) and Gradient Boosting were tried and tuned using cross validation.

```

for n in [50, 100]:
    for lr in [0.5, 1.0]:
        model = AdaBoostClassifier(
            estimator=DecisionTreeClassifier(max_depth=1,
                random_state=42),
            n_estimators=n, learning_rate=lr, random_state=42
        )
        scores = cross_validate(model, X_train, y_train, cv=cv,
            scoring=["accuracy", "f1"])

```

| Model             | n_estimators | learning rate | Avg CV Accuracy (%) | Avg CV F1 Score (%) |
|-------------------|--------------|---------------|---------------------|---------------------|
| AdaBoost          | 50           | 0.5           | 96.48               | 95.12               |
| AdaBoost          | 50           | 1.0           | 96.70               | 95.46               |
| AdaBoost          | 100          | 0.5           | 97.14               | 96.07               |
| AdaBoost          | 100          | 1.0           | 96.48               | 95.23               |
| Gradient Boosting | 50           | 0.1           | 95.60               | 94.09               |
| Gradient Boosting | 100          | 0.1           | 96.70               | 95.53               |

Table 3: Boosting Hyperparameter Evaluation

**Inference:** AdaBoost with `n_estimators=100` and `learning_rate=0.5` gave the best cross-validation accuracy (97.14%), slightly ahead of Gradient Boosting's best configuration (96.70%). This was chosen as the final boosting model. It is interesting that

AdaBoost, using very simple decision stumps as weak learners, was able to outperform Gradient Boosting here.

### 5.3 Stacked Ensemble

The Stacked Ensemble used SVM, Naive Bayes and a Decision Tree as base models, with Logistic Regression as the meta-learner combining their predictions.

```
stacking_model = StackingClassifier(
    estimators=[
        ("svm", svm_pipeline),
        ("nb", GaussianNB()),
        ("dt", DecisionTreeClassifier(max_depth=5, random_state
                                     =42))
    ],
    final_estimator=LogisticRegression(max_iter=1000,
                                       random_state=42),
    cv=5
)
```

| Base Models                       | Meta Learner        | Avg CV Accuracy (%) | Avg CV F1 Score (%) |
|-----------------------------------|---------------------|---------------------|---------------------|
| SVM + Naive Bayes + Decision Tree | Logistic Regression | 96.26               | 94.90               |

Table 4: Stacked Ensemble Evaluation

**Inference:** The Stacked Ensemble’s cross-validation accuracy (96.26%) is actually a bit lower than the best Bagging (96.92%) and Boosting (97.14%) configurations found. This is a bit surprising since stacking is generally expected to do well, but it shows that using more/different models does not automatically guarantee a better result, especially if some of the base models (like plain Gaussian Naive Bayes here) are individually weaker on this dataset.

## 6 Performance Comparison on Test Set

| Model            | Accuracy (%) | Precision (%) | Recall (%) | F1 Score (%) |
|------------------|--------------|---------------|------------|--------------|
| Bagging          | 97.37        | 100.0         | 92.86      | 96.30        |
| Boosting         | 97.37        | 100.0         | 92.86      | 96.30        |
| Stacked Ensemble | 96.49        | 100.0         | 90.48      | 95.00        |

Table 5: Performance Comparison of Ensemble Models

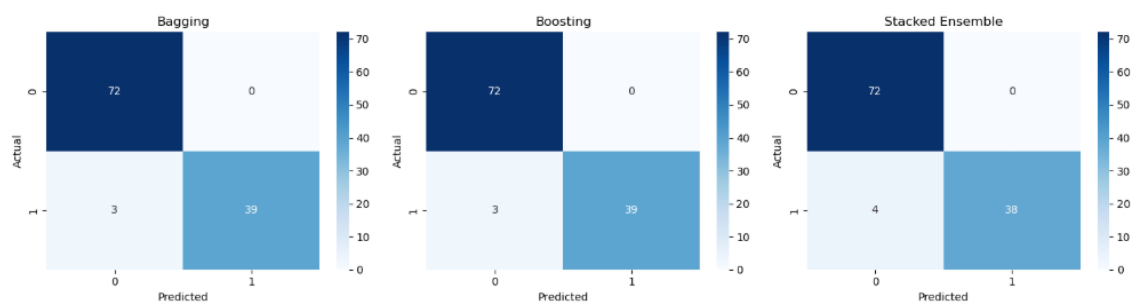


Figure 3: Confusion Matrices - Bagging, Boosting, Stacked Ensemble

**Inference:** All three models achieve perfect precision (100%) on the test set, meaning none of them ever predicted a benign case as malignant. Bagging and Boosting both missed the same number of malignant cases, while the Stacked Ensemble missed slightly more (lower recall of 90.48%). So on this particular test split, Bagging and Boosting were marginally better at actually catching malignant cases.

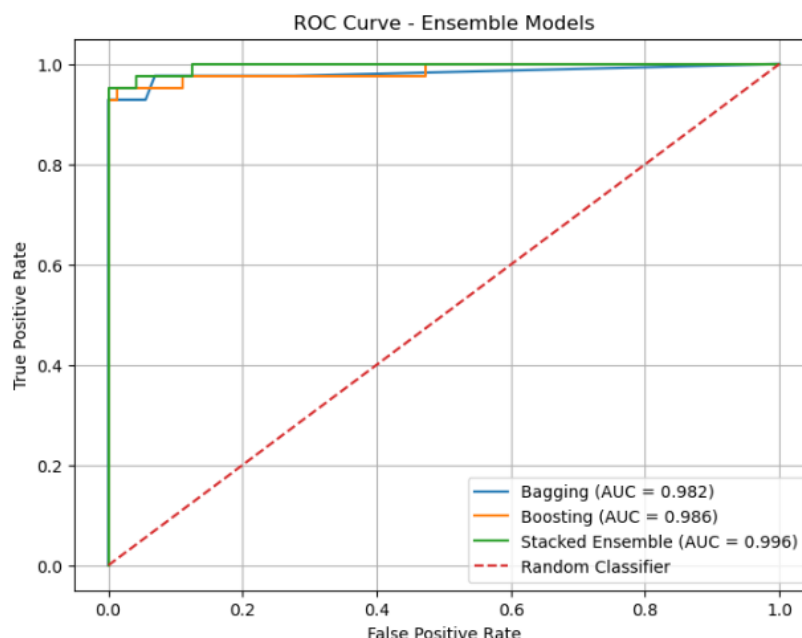


Figure 4: ROC Curve Comparison of Ensemble Models

**Inference:** Interestingly, even though the Stacked Ensemble had slightly lower accuracy and recall at the default 0.5 threshold, it actually has the highest ROC-AUC (0.996) compared to Bagging (0.982) and Boosting (0.986). This means the Stacked Ensemble is doing a very good job of ranking/separating the classes overall, and its performance at the specific 0.5 threshold could possibly be improved further by adjusting the decision threshold.

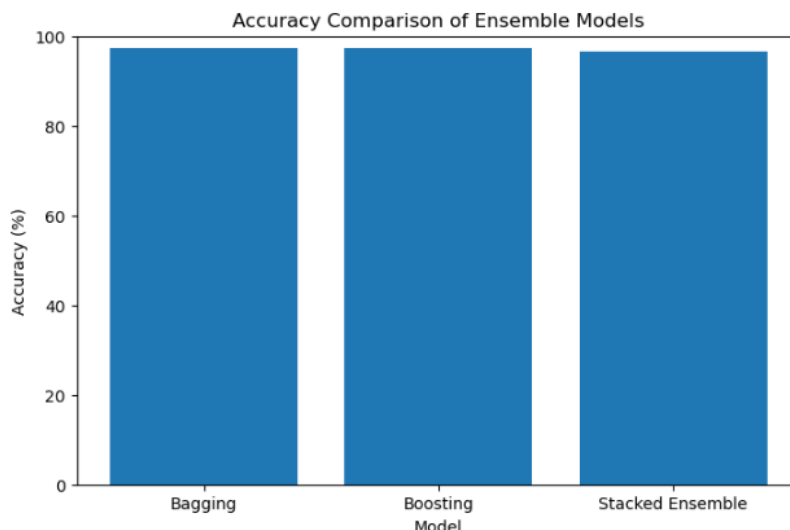


Figure 5: Accuracy Comparison of Ensemble Models

**Inference:** Bagging and Boosting are tied for the highest test accuracy (97.37%), with the Stacked Ensemble slightly behind (96.49%). All three models are quite close to each other though, showing that all three ensemble strategies work reasonably well on this dataset.

## 7 Observation Questions

### 1. How does Bagging reduce variance?

Bagging trains multiple decision trees independently on different bootstrap samples of the training data. Since a single decision tree can be quite sensitive to small changes in the training data (high variance), averaging the predictions of many such trees smooths out this sensitivity, individual errors that are specific to one bootstrap sample tend to cancel out when combined, which reduces the overall variance of the final prediction.

### 2. How does Boosting address model bias?

Boosting builds models sequentially, where each new model is specifically trained to focus more on the samples that previous models got wrong. This means the ensemble keeps actively correcting its own weak points step by step, rather than treating every sample equally like bagging does. This sequential error correction is what allows boosting to reduce bias, since even simple weak learners like decision stumps (as used in AdaBoost here) can be combined into a strong overall model.

### 3. Why does stacking benefit from heterogeneous models?

Different types of models tend to make different kinds of errors, an SVM finds a margin-based boundary, Naive Bayes assumes feature independence, and a Decision Tree splits the feature space into rectangular regions. Since these models look at the data in different ways, their mistakes are less likely to overlap, and a meta-learner can learn to combine their predictions in a way that plays to each model's strengths. If all base models were the same type, they would likely make similar mistakes, which defeats the purpose of combining them.

### 4. Which ensemble method performed best and why?

Based on test set accuracy, Bagging and Boosting were tied at the top (97.37%), slightly ahead of the Stacked Ensemble (96.49%). However, looking at ROC-AUC, the Stacked

Ensemble was actually the best (0.996) among the three. This shows there isn't one single "best" method overall, it depends on the metric being prioritized, AdaBoost/Bagging gave the best accuracy at the default threshold in this experiment, but the Stacked Ensemble showed the strongest class separation ability.

## 8 Conclusion

In this experiment, Bagging (using Decision Trees), Boosting (AdaBoost and Gradient Boosting) and a Stacked Ensemble (SVM + Naive Bayes + Decision Tree with a Logistic Regression meta-learner) were implemented and compared on the breast cancer dataset. Bagging with 25 estimators and AdaBoost with 100 estimators and learning\_rate=0.5 both achieved the best test accuracy (97.37%), while the Stacked Ensemble achieved a slightly lower accuracy but the highest ROC-AUC. This experiment showed how bagging reduces variance through independent averaging, boosting reduces bias through sequential error correction, and stacking can combine diverse models, though it does not automatically guarantee the single best result on every metric. Overall, ensemble methods clearly performed well and were competitive with or better than the individual models seen in earlier experiments on this same dataset.

## 9 References

1. Scikit-learn Documentation: Ensemble Methods. <https://scikit-learn.org/stable/modules/ensemble.html>
2. Scikit-learn Documentation: Bagging Classifier. <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.BaggingClassifier.html>
3. Wisconsin Diagnostic Breast Cancer Dataset, UCI Machine Learning Repository.